

Quiz 4: Mini-Proyecto Integrador

Back-End, Git y Fundamentos Web

Curso: IF0009 - Desarrollo de Software IV

Docente: Mag. Jonathan Granados C.

Puntuación Total: 25 Puntos (Equivalente al 100% de la nota del Quiz 4)

Modalidad: Trabajo Práctico Autónomo Individual

Tiempo Estimado de Ejecución: 2.5 a 3 horas

■ Enunciado General de la Evaluación Práctica

El objetivo de esta evaluación es comprobar la capacidad del estudiantado para integrar y aplicar de manera autónoma los conocimientos adquiridos en los **tres primeros temas del curso**:

1. **Tema 1:** Fundamentos de Desarrollo Web y Arquitecturas Full-Stack.
2. **Tema 2:** Configuración del Ambiente de Desarrollo y Control de Versiones con Git.
3. **Tema 3:** Desarrollo Back-End Completo (Persistencia JPA, Lógica de Negocio, APIs RESTful, Manejo Centralizado de Errores RFC 7807, Seguridad JWT y Pruebas Unitarias con JUnit 5 / Mockito).

En lugar de resolver un cuestionario teórico, usted deberá construir desde cero y entregar un **Mini-Proyecto de Software Back-End** completamente funcional en Java (Spring Boot) configurado bajo arquitectura limpia en capas, versionado en Git y listo para su consumo seguro.

■ Caso de Estudio: Sistema de Gestión de Envíos

"LogiExpress"

La empresa de logística urbana **LogiExpress** requiere automatizar la gestión de sus paquetes y rutas de entrega mediante una API RESTful robusta, segura y comprobada mediante pruebas

automatizadas.

Usted ha sido contratado como Ingeniero de Software Back-End para diseñar y programar la primera versión de la API de **LogiExpress**.

■ Especificaciones Técnicas del Proyecto por Tema Evaluado

Tema 1: Fundamentos Web y Arquitectura de Pila Completa (2 Puntos)

1. Diseñe la aplicación siguiendo el modelo de arquitectura desacoplada cliente-servidor orientada a servicios RESTful.
2. Justifique el desacoplamiento de capas en la documentación del proyecto (README.md), explicando cómo la API REST sirve como columna vertebral para futuros clientes (Web SPA o Móvil).

Tema 2: Configuración del Ambiente y Flujo de Trabajo en Git (3 Puntos)

1. **Inicialización y Rama Principal:** Inicialice el repositorio local de Git sobre la rama principal `main` (`git branch -M main`).
2. **Historial de Commits Semánticos:** El repositorio debe contener un historial mínimo de **5 commits semánticos** siguiendo el estándar `Conventional Commits` (`feat:`, `fix:`, `docs:`, `refactor:`).
3. **Uso de Ramas de Características:** Realice la implementación de la capa de seguridad o de pruebas en una rama secundaria (ej. `feature/security` o `feature/testing`) e intégreala a `main` mediante un `merge`.
4. **Archivo .gitignore:** Incluya un archivo `.gitignore` adecuado que excluya los binarios de compilación (`/target`, `.class`), configuraciones locales del IDE (`.idea`, `.settings`) y claves secretas.

Tema 3: Desarrollo Back-End Completo (15 Puntos)

3.1. Capa de Datos (data) y Persistencia ORM (3 Puntos)

- Implemente dos entidades relacionales JPA con anotaciones de mapeo:
- `Cliente`: `id` (Long, PK autogenerada), `cedula` (String, único), `nombre` (String), `correo` (String).
- `Paquete`: `id` (Long, PK autogenerada), `codigoRastreo` (String, único), `descripcion` (String), `pesoKg` (Double), `estado` (Enum: REGISTRADO, EN_TRANSITO, ENTREGADO), y relación `@ManyToOne` hacia `Cliente`.
- Configure el repositorio Spring Data JPA e incluya un método de consulta personalizado JPQL que permita buscar paquetes por estado retornando resultados paginados (`Pageable` / `Page<Paquete>`).

3.2. Capa de Negocios (business) y Validaciones (3 Puntos)

- Cree el servicio `@Service` denominado `PaqueteService`.
- Anote los métodos de escritura con `@Transactional`.
- Implemente la siguiente regla de negocio obligatoria:
- Si el `pesoKg` del paquete es superior a 30.0 kg, el sistema debe rechazar la operación lanzando una excepción personalizada de negocio `PesoExcedidoException`.
- Si el cliente no existe en la base de datos, lanzar `ClienteNoEncontradoException`.

3.3. Capa de Controladores (controller) y DTOs (3 Puntos)

- Exponga el controlador REST `@RestController` bajo la ruta base `/api/v1/paquetes`.
- Utilice DTOs (`PaqueteRequestDto` y `PaqueteResponseDto`) desacoplando las entidades JPA de la capa HTTP.
- Aplique validaciones `@Valid` en las peticiones (`@NotNull`, `@NotBlank`, `@Positive`).
- Documente los endpoints y sus códigos de respuesta HTTP usando anotaciones de **OpenAPI 3.0 / Swagger UI** (`@Operation`, `@ApiResponse`).

3.4. Manejo Centralizado de Errores con RFC 7807 (2 Puntos)

- Implemente un manejador de excepciones global `@RestControllerAdvice`.

- Capture las excepciones `PesoExcedidoException` (HTTP 400 Bad Request) y `ClienteNoEncontradoException` (HTTP 404 Not Found).
- Retorne la respuesta estructurada bajo el estándar **RFC 7807 (Problem Details)** incorporando los campos: `type`, `title`, `status`, `detail`, `instance` y `timestamp`.

3.5 & 3.6. Seguridad RBAC y Sesiones seguras con JWT (2 Puntos)

- Configure **Spring Security** para proteger las rutas de la API.
- Implemente la emisión y validación de tokens **JWT (RFC 7519)** mediante un endpoint de autenticación `/api/v1/auth/login`.
- Defina dos roles de usuario en el sistema: `ROLE_ADMIN` y `ROLE_OPERADOR`.
- Los usuarios con `ROLE_ADMIN` u `ROLE_OPERADOR` pueden consultar paquetes (GET).
- Solo los usuarios con `ROLE_ADMIN` pueden registrar o eliminar paquetes (POST, DELETE).
- Encripte las contraseñas guardadas usando `BCryptPasswordEncoder`.

3.7. Pruebas Unitarias con JUnit 5 Jupiter y Mockito (2 Puntos)

- Construya una clase de prueba unitaria `PaqueteServiceTest` usando **JUnit 5 Jupiter** y **Mockito**.
- Escriba al menos dos métodos de prueba automatizados:
- Verificar el registro exitoso de un paquete cuando el peso es válido (`assertNotNull`, `verify`).
- Verificar que se lanza la excepción `PesoExcedidoException` cuando el peso supera los 30.0 kg (`assertThrows`).

Entregable del Proyecto y Estructura de Código (5 Puntos)

1. Estructura Multicapa Obligatoria: El proyecto en Java debe respetar estrictamente los siguientes paquetes: `* com.logiexpress.domain` (Entidades del modelo) `* com.logiexpress.data` (Repositorios Spring Data JPA) `* com.logiexpress.business` (Servicios y reglas de negocio) `* com.logiexpress.controller` (Controladores REST y DTOs) `* com.logiexpress.config`

(Seguridad y OpenAPI) * `com.logiexpress.exception` (Manejador RFC 7807 y Excepciones)

2. Versión de Java: Configurar el proyecto en el `pom.xml` con Java 21 (o indicar en el README la versión de JDK local utilizada).

3. Repositorio GitHub: Entregar el enlace al repositorio público de GitHub en la plataforma Mediación Virtual.

■ Rúbrica de Evaluación Sumativa (25 Puntos)

Criterio	Puntos	Nivel Excelente (100%)	Nivel Aceptable (70-99%)	Nivel Deficiente (<70%)
Git y Control de Versiones	3 pts	Repositorio en <code>main</code> , más de 5 commits semánticos, uso de rama secundaria e integración limpia con <code>.gitignore</code> .	Commits con mensajes genéricos o falta la rama secundaria.	Menos de 3 commits, sin <code>.gitignore</code> o sube la carpeta <code>/target</code> .
Persistencia JPA y Capa de Datos	3 pts	Entidades relacionales correctamente mapeadas, <code>@ManyToOne</code> funcional y consulta JPQL paginada.	Relaciones mapeadas pero falla la paginación o la consulta JPQL.	Faltan entidades, mapeos erróneos o sin repositorio funcional.

Lógica de Negocio y Transacciones	3 pts	Servicio <code>@Service</code> desacoplado, reglas de peso validada, excepciones personalizadas y <code>@Transactional</code> .	Regla de negocio implementada directamente en el controlador o sin transacciones.	Sin lógica de validación ni manejo de excepciones de negocio.
API RESTful, DTOs y OpenAPI	3 pts	Endpoints REST bien estructurados, uso de DTOs con <code>@Valid</code> , códigos HTTP correctos y Swagger activo.	Uso de entidades JPA como DTOs o falta documentación OpenAPI.	Endpoints inestables, verbos HTTP incorrectos o sin validaciones.
Manejo de Errores RFC 7807	2 pts	<code>@RestControllerAdvice</code> completo devuelvo el objeto Problem Details bajo RFC 7807 impecable.	Manejador presente pero no cumple la estructura completa del RFC 7807.	Excepciones no capturadas que exponen el stacktrace al cliente.

Seguridad JWT y Roles (RBAC)	2 pts	Autenticación JWT funcional, BCrypt activo y endpoints restringidos según los roles <code>ADMIN</code> u <code>OPERADOR</code> .	JWT emite token pero falla el filtro de autorización o restricción de rol.	Endpoints totalmente públicos sin seguridad ni encriptación.
Pruebas Unitarias (JUnit/Mockito)	2 pts	Mínimo 2 pruebas unitarias pasando exitosamente con Mockito y aserciones de JUnit 5 Jupiter.	Solo 1 prueba implementada o faltan verificaciones con Mockito.	Sin pruebas unitarias o las pruebas fallan en compilación.
Estructura y Documentación	7 pts	Estructura multicapa impecable (domain, data, business, controller), <code>README.md</code> completo con instrucciones de ejecución.	Falta un paquete de la arquitectura requerida o <code>README</code> incompleto.	Proyecto monolítico en un único paquete o no compila.

■ ■ Guía de Referencia Técnica y Solución de Arquitectura (Para el Evaluador)

A continuación se presentan los fragmentos de código de referencia que componen la arquitectura esperada del proyecto evaluado.

1. Modelo de Dominio JPA (`com.logiexpress.domain`)

```
package com.logiexpress.domain;

import jakarta.persistence.*;
import java.io.Serializable;

@Entity
@Table(name = "clientes")
public class Cliente implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false, length = 20)
    private String cedula;

    @Column(nullable = false, length = 100)
    private String nombre;

    @Column(nullable = false, length = 100)
    private String correo;

    public Cliente() {}

    public Cliente(Long id, String cedula, String nombre, String correo) {
        this.id = id;
        this.cedula = cedula;
        this.nombre = nombre;
        this.correo = correo;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getCedula() { return cedula; }
    public void setCedula(String cedula) { this.cedula = cedula; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getCorreo() { return correo; }
    public void setCorreo(String correo) { this.correo = correo; }
}
```

```

package com.logiexpress.domain;

import jakarta.persistence.*;
import java.io.Serializable;

@Entity
@Table(name = "paquetes")
public class Paquete implements Serializable {

    public enum EstadoPaquete { REGISTRADO, EN_TRANSITO, ENTREGADO }

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false, length = 30)
    private String codigoRastreo;

    @Column(nullable = false)
    private String descripcion;

    @Column(nullable = false)
    private Double pesoKg;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private EstadoPaquete estado;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "cliente_id", nullable = false)
    private Cliente cliente;

    public Paquete() {}

    public Paquete(Long id, String codigoRastreo, String descripcion,
        Double pesoKg, EstadoPaquete estado, Cliente cliente) {
        this.id = id;
        this.codigoRastreo = codigoRastreo;
        this.descripcion = descripcion;
        this.pesoKg = pesoKg;
        this.estado = estado;
        this.cliente = cliente;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

```

```

    public String getCodigoRastreo() { return codigoRastreo; }
    public void setCodigoRastreo(String codigoRastreo) {
        this.codigoRastreo = codigoRastreo;
    }
    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }
    public Double getPesoKg() { return pesoKg; }
    public void setPesoKg(Double pesoKg) { this.pesoKg = pesoKg; }
    public EstadoPaquete getEstado() { return estado; }
    public void setEstado(EstadoPaquete estado) { this.estado = estado; }
    public Cliente getCliente() { return cliente; }
    public void setCliente(Cliente cliente) { this.cliente = cliente; }
}

```

2. Capa de Datos (com.logiexpress.data)

```

package com.logiexpress.data;

import com.logiexpress.domain.Paquete;
import com.logiexpress.domain.Paquete.EstadoPaquete;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

@Repository
public interface PaqueteRepository extends JpaRepository<Paquete, Long> {

    @Query("SELECT p FROM Paquete p WHERE p.estado = :estado")
    Page<Paquete> findByEstadoPaginado(@Param("estado") EstadoPaquete estado,
                                       Pageable pageable);
}

```

3. Capa de Negocios (com.logiexpress.business)

```

package com.logiexpress.business;

import com.logiexpress.data.ClienteRepository;
import com.logiexpress.data.PackageRepository;
import com.logiexpress.domain.Cliente;
import com.logiexpress.domain.Package;
import com.logiexpress.exception.ClienteNoEncontradoException;
import com.logiexpress.exception.PesoExcedidoException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class PackageService {

    private final PackageRepository packageRepository;
    private final ClienteRepository clienteRepository;

    public PackageService(PackageRepository packageRepository,
                          ClienteRepository clienteRepository) {
        this.packageRepository = packageRepository;
        this.clienteRepository = clienteRepository;
    }

    @Transactional
    public Package registrarPaquete(Package paquete, Long clienteId) {
        if (paquete.getPesoKg() > 30.0) {
            throw new PesoExcedidoException(
                "El peso del paquete (" + paquete.getPesoKg() +
                " kg) excede el limite maximo permitido de 30.0 kg."
            );
        }

        Cliente cliente = clienteRepository.findById(clienteId)
            .orElseThrow(() -> new ClienteNoEncontradoException(
                "Cliente con ID " + clienteId + " no fue encontrado."
            ));

        paquete.setCliente(cliente);
        return packageRepository.save(paquete);
    }
}

```

4. Manejo Centralizado de Excepciones RFC 7807 (`com.logiexpress.exception`)

```

package com.logiexpress.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.net.URI;
import java.time.Instant;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(PesoExcedidoException.class)
    public ProblemDetail handlePesoExcedido(PesoExcedidoException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, ex.getMessage());
        problem.setTitle("Regla de Negocio Violada: Peso Excedido");
        problem.setType(URI.create("https://logiexpress.com/errors/peso-excedido"));
        problem.setProperty("timestamp", Instant.now());
        return problem;
    }

    @ExceptionHandler(ClienteNoEncontradoException.class)
    public ProblemDetail handleClienteNoEncontrado(ClienteNoEncontradoException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setTitle("Recurso No Encontrado");
        problem.setType(URI.create("https://logiexpress.com/errors/cliente-no-encontrado"));
        problem.setProperty("timestamp", Instant.now());
        return problem;
    }
}

```

5. Pruebas Unitarias con JUnit 5 Jupiter y Mockito

```

package com.logiexpress.business;

import com.logiexpress.data.ClienteRepository;
import com.logiexpress.data.PaqueteRepository;
import com.logiexpress.domain.Cliente;
import com.logiexpress.domain.Paquete;

```

```

import com.logiexpress.domain.Paquete.EstadoPaquete;
import com.logiexpress.exception.PesoExcedidoException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class PaqueteServiceTest {

    @Mock
    private PaqueteRepository paqueteRepository;

    @Mock
    private ClienteRepository clienteRepository;

    @InjectMocks
    private PaqueteService paqueteService;

    private Cliente clienteValido;
    private Paquete paqueteValido;

    @BeforeEach
    void setUp() {
        clienteValido = new Cliente(1L, "112340567", "Juan Perez", "juan@logi.com");
        paqueteValido = new Paquete(null, "PKG-001", "Caja con Libros",
                                     12.5, EstadoPaquete.REGISTRADO, null);
    }

    @Test
    @DisplayName("Registrar Paquete: Exitoso cuando el peso es menor o igual a 30kg")
    void registrarPaquete_Exitoso() {
        when(clienteRepository.findById(1L)).thenReturn(Optional.of(clienteValido));
        when(paqueteRepository.save(any(Paquete.class))).thenReturn(paqueteValido);

        Paquete resultado = paqueteService.registrarPaquete(paqueteValido, 1L);
    }

```

```

        assertNotNull(resultado);
        assertEquals("PKG-001", resultado.getCodigoRastreo());
        verify(clienteRepository, times(1)).findById(1L);
        verify(paqueteRepository, times(1)).save(paqueteValido);
    }

    @Test
    @DisplayName("Registrar Paquete: Falla con PesoExcedidoException si peso > 30kg")
    void registrarPaquete_PesoExcedido_LanzaExcepcion() {
        Paquete paquetePesado = new Paquete(null, "PKG-999", "Maquinaria",
                                              45.0, EstadoPaquete.REGISTRADO, null);

        assertThrows(PesoExcedidoException.class, () -> {
            paqueteService.registrarPaquete(paquetePesado, 1L);
        });

        verify(paqueteRepository, never()).save(any());
    }
}

```